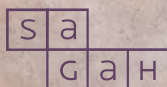


PENSAMENTO COMPUTACIONAL

Cleiton Silvano Goulart



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS



Comando `for`, variáveis contadoras e acumuladoras (utilizando a linguagem Python)

Objetivos de aprendizagem

Ao final deste texto, você deve apresentar os seguintes aprendizados:

- Descrever o comando `for` e as variáveis contadoras e acumuladoras.
- Realizar testes de mesa em problemas que utilizem o comando `for` e as variáveis contadoras e acumuladoras.
- Aplicar o comando `for` e as variáveis contadoras e acumuladoras em soluções computacionais.

Introdução

Neste capítulo, você vai estudar o comando `for` da linguagem Python. Com ele, você poderá criar variáveis contadoras e variáveis acumuladoras, que são muito importantes para manipular alguma variável do tipo lista. Além disso, você vai analisar como funciona o teste de mesa para algoritmos que usam o comando `for`. Esse teste é uma importante ferramenta, que pode ser empregada para testar e validar um algoritmo que foi desenvolvido.

O comando `for` e as variáveis contadoras e acumuladoras

O comando `for` consiste em uma estrutura de repetição baseada em alguma variável do tipo lista. Com esse comando, é possível executar um código específico para cada elemento da lista. A lista pode ser qualquer tipo específico que você já tenha disponível no seu código.

Na Figura 1, você pode ver um exemplo do uso do comando `for`. Nesse exemplo, foi declarada uma variável do tipo lista, contendo do número 10 ao número 15. O comando `for` faz com que a variável `valor` assuma cada um dos elementos da lista, um de cada vez. Dessa forma, o comando `print` é capaz de imprimir cada um dos números da lista (PYTHON SOFTWARE FOUNDATION, 2019).

```
[1]: lista = [10, 11, 12, 13, 14, 15]
    for valor in lista:
        print(valor)

10
11
12
13
14
15
```

Figura 1. Exemplo de uso do comando `for` para imprimir os números de uma lista de valores prévios.

Quando você não tem uma lista, mas, mesmo assim, quer usar o comando `for`, é possível usá-lo com o comando `range`. Veja a Figura 2 para um exemplo dessa abordagem. O comando `range` cria uma lista de valores com cinco números inteiros, começando em 0. Na Figura 2 você também pode observar outro exemplo de uso do comando `for` junto ao comando `range`. Utiliza-se a expressão `range(4, 8)` para que a lista numérica seja iniciada em 4 e vá até o número 8.

```
[1]: for valor in range(5):  
      print(valor)
```

```
0  
1  
2  
3  
4
```

```
[1]: for valor in range(4, 8):  
      print(valor)
```

```
4  
5  
6  
7
```

Figura 2. Exemplo de uso do comando `for` para imprimir os números de uma lista de valores criada com o comando `range`.

Outros tipos de variáveis também podem ser usados junto com o comando `for`. Podemos usar uma variável de texto, ou *string*, junto com o comando `for`; na Figura 3, temos um exemplo dessa aplicação. Repare que o texto nada mais é do que uma lista de caracteres. O comando `for` é capaz de atribuir à variável `caracter` cada letra da variável `texto`, uma de cada vez. Observa-se que o comando `for` é extremamente útil quando queremos percorrer cada elemento de uma lista (PYTHON SOFTWARE FOUNDATION, 2019).

```
[1]: texto = "Python"  
      for caracter in texto:  
          print(caracter)
```

```
P  
y  
t  
h  
o  
n
```

Figura 3. Exemplo de uso do comando `for` para imprimir cada caractere de uma variável do tipo *string*.

Variáveis contadoras e acumuladoras

É muito comum realizarmos alguma operação numérica com listas. Por exemplo, se quisermos contar quantos caracteres existem em uma frase, podemos usar a função `for`, juntamente com uma variável, para a contagem de caracteres. Na Figura 4, temos um exemplo que ilustra exatamente essa situação. Observe que, para esse exemplo, foi criada a variável `qtdeLetras`. Dentro da estrutura do comando `for`, a cada letra que ele percorre na frase, a variável `qtdeLetras` é incrementada, isto é, é acrescida uma unidade para seu valor. Pode-se dizer que essa variável é uma **variável contadora**. Repare que a variável contadora não considera o valor da variável de iteração do comando `for`, a qual, nesse exemplo, é a variável `letra`.

```
[1]: frase = "A ligeira raposa marrom ataca o cão preguiçoso"
qtdeLetras = 0
for letra in frase:
    qtdeLetras += 1
print("A frase possui " + str(qtdeLetras) + " letras.")
A frase possui 16 letras.
```

Figura 4. Exemplo do uso do comando `for` para fazer a contagem de caracteres de uma frase.

Às vezes, precisamos considerar o valor da variável de iteração do comando `for`. Um exemplo clássico é o algoritmo para o cálculo de uma média, que está ilustrado na Figura 5. Observe que a variável `soma` vai aumentando seu valor a cada iteração do comando `for`, sempre se somando com o valor da variável de iteração do comando `for`. A variável `soma` é uma **variável acumuladora**, pois ela está, a cada iteração, acumulando o seu valor com o valor da variável de iteração do comando `for`.

```
[1]: dados = [1, 3, 5, 8, 10, 2]

soma = 0
qtde = 0

for valor in dados:
    soma += valor
    qtde += 1

print(soma)
print(qtde)

media = soma / qtde

print(media)

29
6
4.833333333333333
```

Figura 5. Exemplo do uso do comando `for` para fazer o cálculo da média de uma lista de valores numéricos.

Apesar de seu uso simples, as variáveis contadoras e as variáveis acumuladoras podem promover vários erros no algoritmo. A fim de evitar que esses erros ocorram, é necessário sempre fazer o **teste de mesa**.

Testes de mesa com estrutura de repetição `for`

Durante o desenvolvimento dos algoritmos, é necessário que sejam feitos testes, a fim de certificar que o funcionamento corresponde ao esperado. O teste que é realizado durante a fase de desenvolvimento do algoritmo é conhecido como **teste de mesa**, conforme leciona Schach (2009). Ele é realizado geralmente à mão, para cada iteração do comando `for`. É muito importante observar o valor atribuído a cada variável durante cada iteração; dessa forma, é possível acompanhar a evolução da variável sob análise.

Teste de mesa para uma variável acumuladora

As variáveis acumuladoras apresentam uma alteração do seu valor a cada iteração do comando `for`. Dessa forma, quando for realizado um teste de mesa que envolva uma variável acumuladora, é imperativo que seja tomado nota de seu valor a cada iteração. Caso o algoritmo apresente alguma falha, por meio da análise da evolução da variável acumuladora, será possível determinar em qual local do código se encontra a falha de programação.

Na Figura 6, temos um algoritmo simples que soma os números pares até 10. Como a condição de paridade de um número é que ele seja divisível por 2, esse algoritmo deve ainda fazer essa verificação. Ele a faz por meio do comando

`if`. Lembrando que o operador `%` retorna o resto inteiro da divisão de dois números. Caso o número seja par (divisível por 2), ele será somado na variável

acumuladora `soma`. Para verificar se esse algoritmo está funcionando como esperado, vamos realizar um teste de mesa para ele.

```
[1]: soma = 0

for numero in range(10):
    if (numero % 2) == 0:
        soma += numero
```

Figura 6. Algoritmo para somar os números pares até 10.

No Quadro 1, está registrado o valor de cada variável em cada iteração. Observe que a variável `soma` só é alterada quando a variável `numero` assume um valor par.

Quadro 1. Valores para o teste de mesa do algoritmo da Figura 6

Iteração	Variável numero	Variável soma
1	0	0
2	1	0
3	2	2
4	3	2
5	4	6
6	5	6
7	6	12
8	7	12
9	8	20
10	9	20

Teste de mesa com variáveis contadoras

O teste de mesa envolvendo variáveis contadoras é um pouco mais simples do que com variáveis acumuladoras. O valor das variáveis contadoras está diretamente associado ao número de interações que o laço `for` realiza. Um cuidado especial deve ocorrer quando a variável contadora estiver associada a um comando condicional `if`. Nesse caso, a análise da condição deve ser feita com bastante atenção, porque o valor da variável contadora está diretamente associado a essa condição.

Na Figura 7 temos um algoritmo capaz de contar quantos números múltiplos inteiros de 3 existem nos 20 primeiros números naturais. Para isso, foram utilizadas duas variáveis contadoras: `noVerificados` e `noMultiplos`. Na variável `noVerificados`, deverá ser feita a contagem para determinar quantos números foram verificados durante todo o algoritmo. Repare que essa variável não é condicionada. Já a variável `noMultiplos` será incrementada apenas se a condição do comando `if` for satisfeita; assim, essa variável vai contar o número de múltiplos de 3 que existem dentro do intervalo de análise do algoritmo.


```
[1]: noVerificados = 0
    noMultiplos = 0

    for numero in range(20):
        noVerificados += 1
        if (numero % 3) == 0:
            noMultiplos += 1

    print(noVerificados)
    print(noMultiplos)

20
7
```

Figura 7. Algoritmo para contar quantos múltiplos inteiros de 3 existem nos 20 primeiros números naturais.

Para fazer o teste de mesa do algoritmo da Figura 7, foi usada a tabela do Quadro 2 como apoio. Conforme o comando percorria o intervalo de valores definidos pelo comando `range(20)`, ambas as variáveis tiveram seu valor anotado. Os resultados obtidos pela execução do algoritmo, conforme já ilustrado na Figura 7, é idêntico ao resultado do teste de mesa, que foi relacionado no Quadro 2.

Quadro 2. Valores para o teste de mesa do algoritmo da Figura 7

Número	noVeri- ficados	noMul- tiplos	Número	noVeri- ficados	noMul- tiplos
0	1	1	10	11	4
1	2	1	11	12	4
2	3	1	12	13	5
3	4	2	13	14	5
4	5	2	14	15	5
5	6	2	15	16	6
6	7	3	16	17	6
7	8	3	17	18	6
8	9	3	18	19	7
9	10	4	19	20	7

Aplicações práticas das variáveis contadoras e acumuladoras

O comando `for` é uma das estruturas mais básicas existentes na linguagem Python. Ao mesmo tempo que ela é simples, é muito poderosa. Com esse comando e com o auxílio das variáveis contadoras e das variáveis acumuladoras, é possível resolver vários problemas computacionais, conforme expõe Santos (2018).

Cálculo do fatorial de um número

O cálculo do fatorial de um número qualquer consiste em uma operação matemática simples. Diz-se que o fatorial de um número é igual ao produto de todos os números decrescentes do número até 0, isto é, o fatorial $n! = n \cdot (n - 1) \cdot (n - 2) \cdot \dots \cdot (n - k)$, sendo que $k = n - 1$. Por exemplo, o fatorial de 3 é $3! = 3 \cdot 2 \cdot 1 = 6$. Para implementar esse processo na linguagem Python, por meio do comando `for`, pode ser utilizado o código ilustrado na Figura 8.

```
[1]: numero = int(input("Digite o número que se deseja determinar o fatorial: "))  
  
fatorial = 1  
for termo in range(1, (numero + 1)):  
    fatorial *= termo  
  
print("O fatorial de " + str(numero) + "! é : " + str(fatorial))  
  
Digite o número que se deseja determinar o fatorial: 3  
O fatorial de 3! é : 6
```

Figura 8. Exemplo do uso do comando `for` para fazer o cálculo do fatorial de um número qualquer.

No exemplo da Figura 8, o comando `range` teve seus parâmetros alterados para que a contagem não incluísse o número 0. Repare no exemplo da Figura 2, em que o comando `range` inclui o 0, caso não seja informado explicitamente o início e o fim da faixa de valores. Para determinar o valor do fatorial, foi empregado o operador de atribuição composto `*=`, para fazer o produto da variável `fatorial` com cada termo, até chegar no número que o usuário escolheu por meio do comando `input`. O comando `int` serve apenas para converter o valor digitado pelo usuário para um número inteiro.

Soma de duas listas

Uma lista numérica pode ser somada à outra de igual dimensão, por meio da soma de cada elemento individual. Por exemplo, se tivermos uma lista $A = [2, 3, 4]$ e quisermos somar com outra lista $B = [7, -3, 2]$, basta fazer a soma elemento a elemento. Dessa forma, a soma dessas duas listas é dada por: $A + B = [(2 + 7), (3 + (-3)), (4 + 2)] = [9, 0, 6]$, conforme lecionam Forbellone e Eberspächer (2005).

Para fazer essa mesma operação usando a linguagem Python, podemos usar o comando `for` para fazer o controle iterativo, elemento a elemento. Na Figura 9 está ilustrado esse mesmo exemplo, porém, em linguagem Python. Observe que foi utilizada uma variável auxiliar, denominada `indice`, juntamente com o comando `range`. O comando `range`, nesse caso, cria uma lista de números inteiros iniciada em 0, com um total de três elementos. A variável `indice` é usada, nesse caso, para representar o índice de cada lista, que será obtido para fazer a soma. Por exemplo, quando `indice = 0`, o algoritmo estará obtendo o elemento 0 da lista A e o elemento 0 da lista B . Por fim, a lista C será alimentada com o resultado dessa operação.

```
[1]: A = [2, 3, 4]
      B = [7, -3, 2]

      C = []

      for indice in range(3):
          C.append(A[indice] + B[indice])

      print(C)

      [9, 0, 6]
```

Figura 9. Exemplo do uso do comando `for` para fazer a soma de duas listas de tamanhos iguais.

Encontrando números primos

Número primo é todo aquele que é divisível apenas por 1 e por si mesmo. Por exemplo, o número 7 é primo, porque só conseguimos dividir 7 por 7 e por 1 para obter um resultado inteiro, ou uma divisão sem resto. Na computação, os números primos são muito importantes, porque, a partir deles, são construídos vários algoritmos de criptografia. Quando o número é pequeno, é fácil descobrir se ele é ou não um número primo; porém, quanto maior o número fica, mais difícil é descobrir se ele é ou não primo, conforme apontam Holanda Filho e Gomes (2019).



Link

No *link* a seguir, você poderá verificar porque os números primos são tão importantes para a criptografia.

<https://qr.go.page.link/h9Pdd>

É possível usar o comando `for` para criar um algoritmo que busque os números primos dentro de um intervalo de valores. Na Figura 10, está ilustrado um algoritmo que usa dois comandos `for` para essa tarefa. O primeiro comando `for` percorre uma lista de 20 números iniciada em 0, que é produzida pelo comando `range`. Assim, esse algoritmo vai verificar quais são os números primos que existem até o número 20. O segundo comando `for` faz o teste de divisão com todos os números de 1 até o próprio número, para verificar se existe ou não resto na divisão.

Observe que foi utilizado o comando `range`, percorrendo de 1 até o número + 1. Isso é necessário porque o comando `range` cria uma lista iniciada em 0, e não é possível dividir nenhum número por 0. Em seguida, a variável `div` funciona como uma variável contadora, armazenando a quantidade de números que conseguiram dividir o número sob teste, resultando em um resto 0 (o operador `%` retorna o resto inteiro da divisão). Caso o número de divisores seja igual a 2, o número pode ser considerado primo.

```
[1]: numPrimos = []  
  
    for numero in range(20):  
        div = 0  
        for divisor in range(1, numero + 1):  
            if (numero % divisor) == 0:  
                div += 1  
        if div == 2:  
            numPrimos.append(numero)  
  
    print(numPrimos)  
  
[2, 3, 5, 7, 11, 13, 17, 19]
```

Figura 10. Exemplo do uso do comando `for` para determinar números primos.



Referências

FORBELLONE, A. L. V.; EBERSPÄCHER, H. F. *Lógica de programação: a construção de algoritmos e estruturas de dados*. 3. ed. São Paulo: Pearson Prentice Hall, 2005. 218 p.

HOLANDA FILHO, I. O.; GOMES, R. L. R. Criptografia e números primos: um namoro que deu certo. *Portal Educação*, São Paulo, 22 abr. 2014. Disponível em: <https://www.portaleducacao.com.br/conteudo/artigos/pedagogia/criptografia-e-numeros-primos-um-namoro-que-deu-certo/56347>. Acesso em: 9 jun. 2019.

PYTHON SOFTWARE FOUNDATION. *The Python tutorial*. Wilmington, 2019. Disponível em: <https://docs.python.org/3/tutorial/index.html>. Acesso em: 9 jun. 2019.

SANTOS, M. G. *Algoritmos e programação*. Porto Alegre: Sagah, 2018. 89 p.

SCHACH, S. R. *Engenharia de software: os paradigmas clássico & orientado a objetos*. 7. ed. São Paulo, McGraw-Hill, 2009. 618 p.

Leituras recomendadas

DASGUPTA, S.; PAPADIMITRIOU, C.; VIZIRANI, U. *Algoritmos*. São Paulo: McGraw-Hill, 2009. 336 p.

SANTOS, M. G.; SARAIVA, M. O.; GONÇALVES, P. F. *Linguagem de programação*. Porto Alegre: Sagah, 2018. 204 p.

Encerra aqui o trecho do livro disponibilizado para esta Unidade de Aprendizagem. Na Biblioteca Virtual da Instituição, você encontra a obra na íntegra.

Conteúdo:



SOLUÇÕES
EDUCACIONAIS
INTEGRADAS